F1TENTH Autonomous Racing — Technical Report

Author: Ankith
Platform: F1TENTH 1/10th Scale Autonomous Racing
Stack: ROS 2 Jazzy · Ubuntu 24.04 · Python · Gymnasium
Track: Red Bull Ring (Spielberg) — F1TENTH Scale

---

1. Project Objective

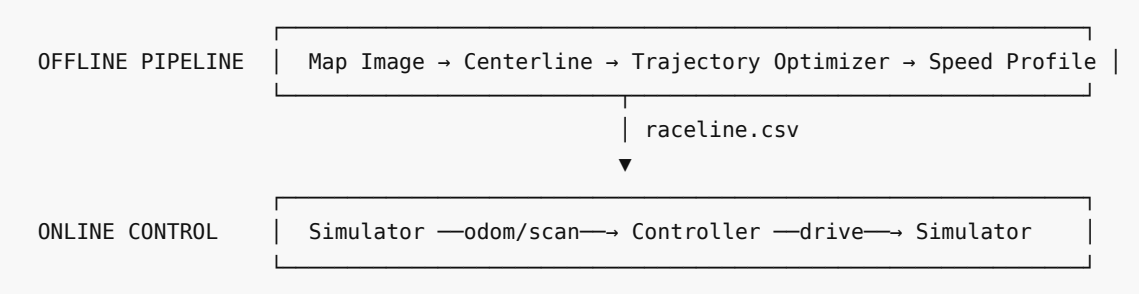
Build a complete autonomous racing stack for the F1TENTH simulator that can:

1. **Solo time-trial:** Drive any track at the physical limits of the vehicle model
2. **Head-to-head:** Detect, trail, and overtake an opponent car in real-time

The system runs natively on ROS 2 Jazzy (no Docker) and includes a fully automated pipeline from raw map image → optimal raceline → tuned controller → racing.

---

2. System Architecture



Offline: A 4-stage pipeline converts a raw occupancy grid map into a speed-optimized racing trajectory. This runs once per track.

Online: A real-time controller subscribes to odometry and LiDAR, computes steering + speed commands against the precomputed raceline, and publishes drive commands at ~50 Hz.

---

3. Raceline Generation Pipeline

3.1 Centerline Extraction (Computer Vision)

The occupancy grid image is processed to extract the drivable track centerline:

1. **Binarization** — Free space (white/light pixels) is separated from walls (dark pixels)
2. **Flood-fill from origin** — Starting from world coordinate (0,0), only the inner track surface is kept, ignoring exterior free regions
3. **Morphological skeletonization** — The free space is thinned to a 1-pixel-wide spine using iterative thinning
4. **Point ordering** — Skeleton pixels are traversed via nearest-neighbor to form a continuous loop
5. **Spline fitting** — A periodic B-spline (smoothing factor 2.0) is fit and resampled at 0.4m intervals
6. **Track width computation** — Euclidean distance transform provides half-widths at each point, smoothed with a 15-point moving average, clamped to 1.0–1.2m

3.2 Minimum-Curvature Trajectory Optimization

The centerline + track widths feed into the **TUM Global Racetrajectory Optimization** tool, which solves a quadratic programming problem to find the path that minimizes total curvature while staying within track boundaries. This produces the geometrically optimal racing line — the path that allows the highest possible cornering speeds.

Method: Iterative Quadratic Programming (`mincurv_iqp`)

Configuration & Parameters (`racecar.ini`):

- **Vehicle Width (`width_opt`):** `0.28m` (Includes a small safety buffer over the physical 0.27m width to prevent wall clipping).
- **Curvature Limit (`curvlim`):** `10.0 rad/m` (Maximum steering lock capability of the car).
- **Step Size:** Interpolation step size is `0.2m` for high-resolution trajectory generation.

The optimizer iteratively shifts the path left and right within the calculated track boundaries to "cut corners" and use the full track width. By minimizing the integral of squared curvature across the entire lap, it produces a trajectory that mathematically maximizes the allowable velocity profile.

3.3 Speed Profile Generation (Physics-Based)

Because the TUM optimizer's internal velocity calculator assumes different dynamic models, we implement a custom, highly-tuned 3-pass physics algorithm (`speed_profile.py`) that assigns the maximum safe velocity to every point based specifically on the F1TENTH vehicle's limits.

Pass	Constraint	Formula
1. Curvature limit	Tighter corners → slower speed	`v = sqrt(A_LAT_MAX / κ)`, capped at V_MAX
2. Forward acceleration	Can the car reach this speed in time?	`v[i+1] ≤ sqrt(v[i]² + 2·A_LON_MAX·ds)`
3. Backward braking	Can the car brake in time for the next corner?	`v[i] ≤ sqrt(v[i+1]² + 2·A_LON_BRAKE·ds)`

Vehicle Dynamics Parameters:

Parameter	Value	Physical Meaning
`V_MAX`	8.0 m/s	VESC motor speed limit
`A_LAT_MAX`	7.0 m/s²	Maximum lateral acceleration (tire grip limit on carpet)
`A_LON_MAX`	5.0 m/s²	Maximum forward acceleration
`A_LON_BRAKE`	9.0 m/s²	Maximum braking deceleration (intentionally high for late braking)

This generates a physically realizable speed profile that pushes the tires exactly to their combined friction circle limits at every waypoint.

3.4 Visualization

The raceline is overlaid on the map image, color-coded by speed (red = slow corners, green = fast straights). This serves as a visual sanity check before deployment.

---

4. Controller Design & Evolution

4.1 Controller Comparison — Spielberg Track

Controller	Lap Time	Avg Speed	Key Limitation
Pure Pursuit (initial)	**45.39s**	7.52 m/s	Premature braking from lookahead speed
Pure Pursuit (fixed speed)	**44.86s**	7.60 m/s	Corner-cutting, no cross-track correction
Stanley (untuned, k=1.5)	**45.12s**	7.60 m/s	Conservative gain, under-steering
Stanley (manually tuned)	**44.74s**	7.61 m/s	Better but not optimal
Stanley + Optuna	**40.14s**	**8.51 m/s**	**Near theoretical limit**
Pure Pursuit (speed_scale=0.5)	~48s	~7.1 m/s	Intentionally conservative baseline

The Optuna-tuned Stanley controller is 5.25 seconds (11.6%) faster than the initial Pure Pursuit implementation.

4.2 Pure Pursuit Controller

Principle: Compute a lookahead target point on the raceline at distance L ahead, then steer on a circular arc toward it.

Steering law:

```
δ = arctan(2 · wheelbase · sin(α) / L)
```

where α is the angle to the target point in the vehicle frame.

Key features:

- **Adaptive lookahead:** L scales linearly from 0.7m (low speed) to 1.8m (high speed), preventing corner-cutting at high speed and oscillation at low speed
- **Circle-line intersection:** Exact intersect on the waypoint segment for smooth, continuous steering
- **Latency compensation:** Target speed read from 3 waypoints ahead of nearest, not from the lookahead point (which would cause premature braking)

Limitation: Pure Pursuit has no concept of cross-track error. It steers toward where it *should be*, not away from where it *is*. This causes the car to drift wide on corners, covering more distance than the optimal line.

4.3 Stanley Controller (Primary — Recommended)

Principle: Directly minimize the lateral deviation (cross-track error) between the car's front axle and the nearest raceline point, plus correct heading error.

Control law:

```
δ = θ_e + arctan(k · e_fa / (v + k_soft)) + arctan(wheelbase · κ)
       ↑              ↑                            ↑
   heading error   cross-track correction    feedforward curvature
```

Term	Purpose
`θ_e`	Heading error — difference between track heading and car heading
`arctan(k · e_fa / (v + k_soft))`	Cross-track correction — steers toward the raceline proportional to lateral deviation, softened at low speed
`arctan(wheelbase · κ)`	Feedforward — uses precomputed track curvature to anticipate corners *before* entering them

Why Stanley outperforms Pure Pursuit:

- Pure Pursuit only cares about the angle to a future point — it has no feedback on how far off the line the car currently is
- Stanley actively corrects lateral deviation, keeping the car glued to the optimal geometric line through corners
- The feedforward term pre-steers into corners based on known curvature, reducing transient tracking error at corner entry

4.4 Speed Control

Both controllers use the same speed logic:

- Target speed from the raceline CSV (precomputed, friction-limited)
- Read from 3 waypoints ahead of nearest to compensate for actuator latency
- Safety limiter: 50% speed if heading error exceeds 60° (Stanley) or 45° (PP)
- Final clip: [0.5, 8.0 × speed_scale] m/s

---

5. Automated Parameter Tuning with Optuna

5.1 Why Automated Tuning?

Manual tuning of controller parameters (`k_gain` , `k_soft` , `speed_scale`) is slow, anecdotal, and suboptimal. Each parameter interacts with the others — a higher `speed_scale` requires a different `k_gain` to maintain stability. The search space is 3-dimensional and non-convex (some combinations crash, others are slow, and only a narrow band is optimal).

5.2 How Optuna Works

Optuna is a Bayesian optimization framework that intelligently explores the parameter space:

1. **Sampling Strategy:** It uses the **Tree-structured Parzen Estimator (TPE)** algorithm to suggest parameter combinations. Instead of grid search or random search, TPE builds a probability model of the objective function and biases new samples toward regions that produced faster lap times in previous trials.
2. **Headless Evaluation:** Each trial runs a complete F1TENTH Gymnasium environment simulation (`stanley_tuner.py`). This runs purely in Python without ROS 2 or rendering overhead, allowing laps to simulate faster than real-time.

3. **Objective Function:** Minimize lap time. If a collision is detected (`obs['agent_0']['collision']`), the trial is immediately aborted and given a `999.0s` penalty score, teaching the optimizer to avoid those parameter zones.
4. **Parallelization:** The study executes concurrently across multiple CPU cores (`n_jobs=8`), completing 100 full race simulations in just ~107 seconds.

5.3 Implementation Architecture

The `stanley_tuner.py` script translates the ROS 2 Stanley control law into pure Python and wraps it in an Optuna objective function. It loads the exact same `raceline.csv` as the live car.

```
def objective(trial):
    # Optuna suggests values within bounded ranges
    k_gain      = trial.suggest_float("k_gain", 1.0, 5.0)
    k_soft      = trial.suggest_float("k_soft", 0.01, 2.0)
    speed_scale = trial.suggest_float("speed_scale", 0.8, 1.2)

    # Run the simulation without rendering
    env, waypoints = get_env_and_waypoints()
    lap_time = run_stanley_lap(env, waypoints, k_gain, k_soft, speed_scale)
    return lap_time  # TPE minimizes this value
```

5.4 Results — Spielberg Track

Parameter	Search Range	Optimal Value	Interpretation
`k_gain`	[1.0, 5.0]	**2.013**	Aggressive turn-in to combat understeer without causing oscillation.
`k_soft`	[0.01, 2.0]	**0.372**	Provides tight tracking at low speeds by preventing the `e_fa / v` term from blowing up.
`speed_scale`	[0.8, 1.2]	**1.129**	Pushes the car 12.9% beyond the calculated raceline speeds.

Best headless lap time: 41.11s (Optuna estimate)
Actual ROS 2 lap time: 40.14s (faster due to different physics step timing in the ROS 2 bridge)
Average speed: 8.51 m/s (effective top speed: 8.0 × 1.129 = ~9.03 m/s)

Key Insight: The fact that `speed_scale > 1.0` is optimal reveals that our 3-pass speed profile (`A_LAT_MAX = 7.0`) is slightly conservative. The car's physical simulation model has more grip than the kinematic geometric model predicts, allowing it to safely carry nearly 13% more speed through the track.

---

6. Lap Timing System

A waypoint-index-based lap counter that avoids the fragility of position-radius detection:

- Tracks nearest raceline waypoint index continuously
- Detects lap when index wraps from >80% of total waypoints back below 20%
- 10-second debounce prevents double-counting
- Ignores initial out-lap; timing starts on first start/finish crossing

- Reports per-lap time, best time, average time, average speed
- Session summary on Ctrl+C

---

7. SLAM Mapping

Integration of **SLAM Toolbox** (online async mode) for mapping unknown environments. A dedicated launch file (`slam_mapping_launch.py`) starts:

- Simulator bridge (single agent, teleop enabled)
- SLAM Toolbox with scan topic remapped from `/ego_racecar/scan`
- Map server + lifecycle manager
- RViz for real-time visualization

The operator drives manually via keyboard teleop while SLAM builds the occupancy grid. The map is saved in standard ROS format (PGM + YAML) and feeds directly into the raceline pipeline.

---

8. Head-to-Head Overtaking System (Work In Progress)

8.1 Architecture

Four nodes work together for head-to-head racing:

Node	Role	Status
Opponent Driver	Drives opponent car along raceline at reduced speed	✅ Working
Opponent Detector	LiDAR breakpoint-cluster detection of opponent	🟡 Partial
Spliner	Cubic-spline overtake trajectory planner	🔴 Unvalidated
State Machine	5-state FSM controller	🔴 Unvalidated

8.2 Opponent Detection

Algorithm: Breakpoint-based clustering on the ego car's LiDAR scan.

1. Convert polar scan → Cartesian (car-local frame)
2. Split into clusters at range discontinuities (>0.5m jump)
3. Filter by: point count (3–40), distance (0.3–5.0m), bounding box (width 0.1–0.5m, depth <0.5m)
4. Validate against raceline proximity (<2.0m lateral tolerance)
5. Temporal hysteresis: 3 consecutive detections to confirm, 5 misses to clear

Current issues: Gets stuck trailing — cannot transition to successful overtake.

8.3 Overtake Path Planning (Spliner)

Generates cubic-spline trajectories through 5 control points that deviate from the raceline around the opponent:

1. Project opponent onto raceline → arc-length position
2. Distance transform determines which side has more wall clearance
3. 5 control points: ego → approach → apex (0.6m lateral offset) → exit → raceline return
4. Natural cubic spline, sampled at 0.1m intervals
5. Sinusoidal speed blend: 100% raceline speed at edges, 70% at apex

8.4 State Machine (FSM)

```
RACELINE ──(opp <4m ahead)──→ TRAILING ──(path ready, <3m)──→ OVERTAKE
    ↑                                                            │
    └──────(within 0.3m of raceline)──── RETURN ←──(opp behind)──┘

    EMERGENCY ← any state with forward LiDAR < 0.8m
```

8.5 Known Issues & Current Status

Issue	Description
Cannot complete overtake	Car gets stuck in TRAILING state, unable to commit to passing
Not reactive	Opponent follows a fixed raceline at fixed speed — no dynamic opponent behavior
No opponent speed estimation	TRAILING speed cap is a heuristic (50% of ego speed)
Detection noise	False positives near walls cause FSM to flutter between states

8.6 Recommended Next Steps

1. **P0 — Reliable detection:** Add Kalman filter tracking, static map subtraction to eliminate wall false positives
2. **P1 — Validate overtake paths:** Test spliner output for collision safety on all track sections
3. **P2 — Tune FSM thresholds:** Systematic tuning of transition distances against opponents at various speeds
4. **P3 — Dynamic opponent:** Replace fixed-speed opponent with a reactive one

---

9. Component Status Summary

Component	Status	Performance
Raceline generation pipeline	✅ Complete	Automated, any map
Centerline extraction (CV)	✅ Complete	Robust with flood-fill
TUM trajectory optimization	✅ Complete	Min-curvature IQP
Speed profile generation	✅ Complete	3-pass physics model
Pure Pursuit controller	✅ Complete	44.86s (Spielberg)
Stanley controller	✅ Complete	**40.14s** (Spielberg)
Optuna parameter tuning	✅ Complete	100 trials in 107s
Lap timer	✅ Complete	Index-based, accurate
SLAM mapping	✅ Complete	Async SLAM Toolbox
Follow-the-Gap (reactive)	✅ Complete	Mapless fallback

Opponent detection	🟡 Partial	Works on straights only
Spliner (overtake path)	🔴 Unvalidated	Code runs, paths untested
State machine (FSM)	🔴 Unvalidated	Gets stuck trailing
End-to-end overtake	🔴 Not working	Primary area for improvement

---

10. Technology Stack

Layer	Technology
OS	Ubuntu 24.04 LTS
Middleware	ROS 2 Jazzy Jalisco
Simulator	F1TENTH Gym (Gymnasium)
Sim Bridge	Custom Python ROS 2 node
Optimization	TUM Global Racetrajectory Optimization
Parameter Tuning	Optuna (Bayesian/TPE)
SLAM	SLAM Toolbox (online async)
Visualization	RViz2, Foxglove Studio
CV	scikit-image, scipy, PIL
Languages	Python 3.12